



Sichuan University -  
Pittsburgh Institute

# ECE0202 Embedded Processors & Interfacing + Lab

Sichuan University-Pittsburgh Institute



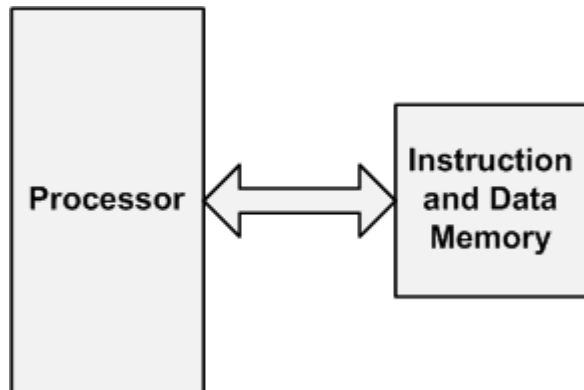
# Topics

- Computer Architecture
  - CPU, Register, Memory, I/O
- How is a program stored?
  - Memory Map
- How does a program run?

# Computer Architecture

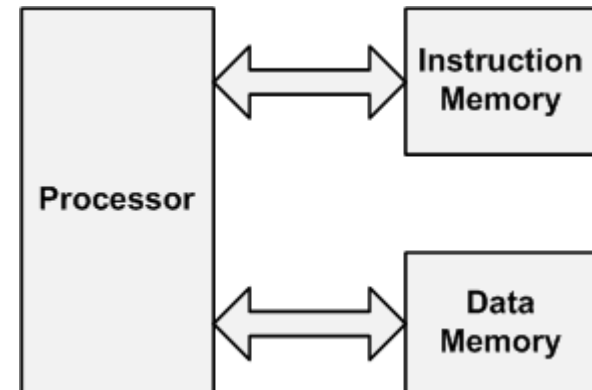
## Von-Neumann

**Instructions and data are stored in the same memory.**



## Harvard

**Data and instructions are stored into separate memories.**



# ARM Cortex Processors

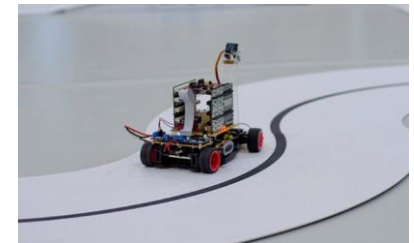
- ARM Cortex-**A** family:
  - **A**pplications processors
  - Support OS and high-performance applications
  - Such as Smartphones, Smart TV



- ARM Cortex-**R** family:
  - **R**eal-time processors with high performance and high reliability
  - Support real-time processing and mission-critical control



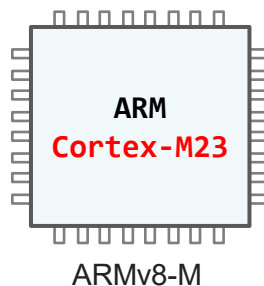
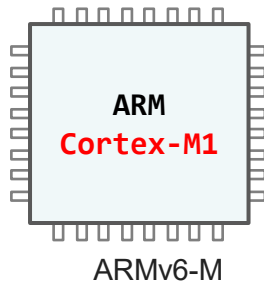
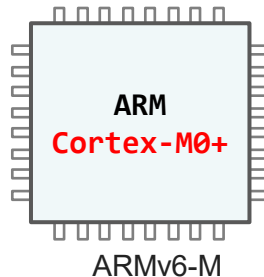
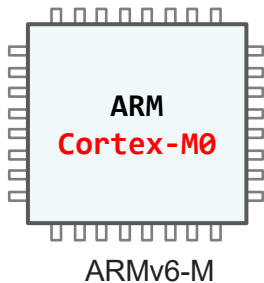
- ARM Cortex-**M** family:
  - **M**icrocontroller
  - Cost-sensitive, support SoC



# ARM Cortex-M Series Family

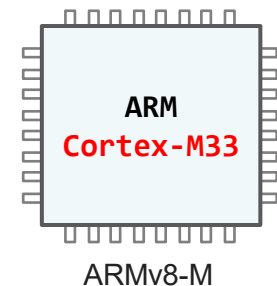
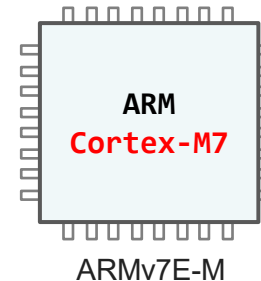
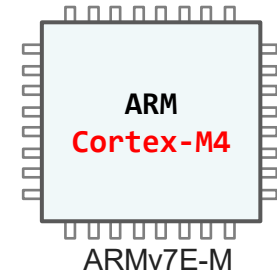
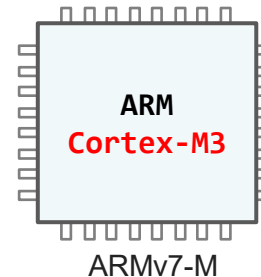
## Von-Neumann

**Instructions and data are stored in the same memory.**



## Harvard

**Data and instructions are stored into separate memories.**

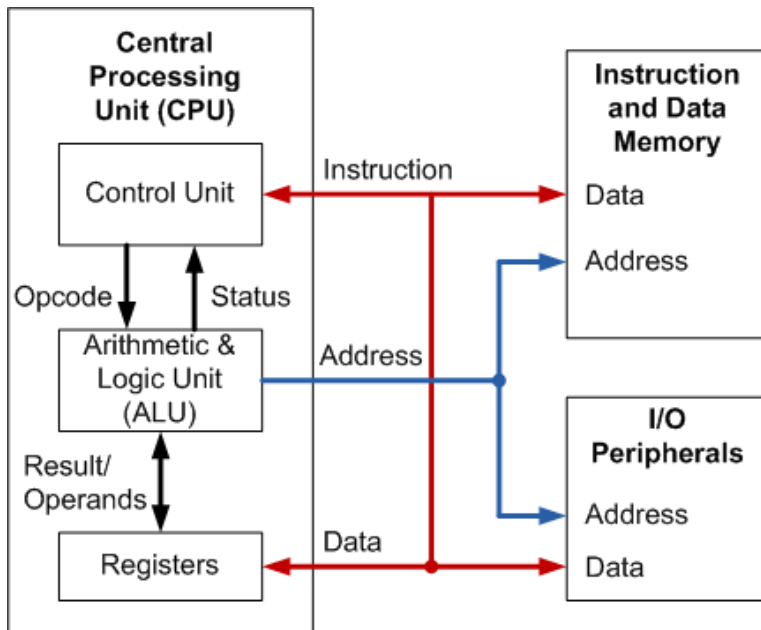


STM32L476RGTx?

# Computer Architecture

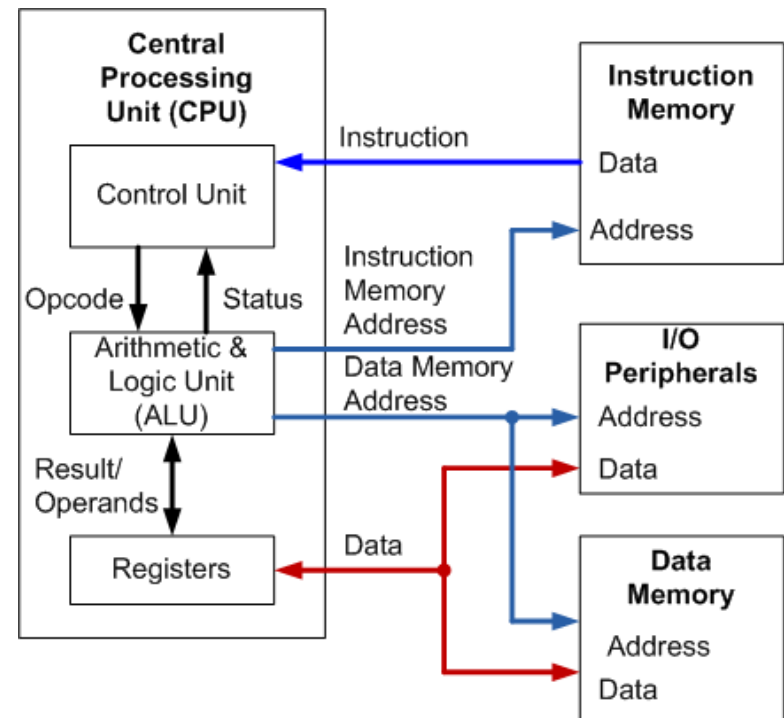
## Von-Neumann

**Instructions and data are stored in the same memory.**



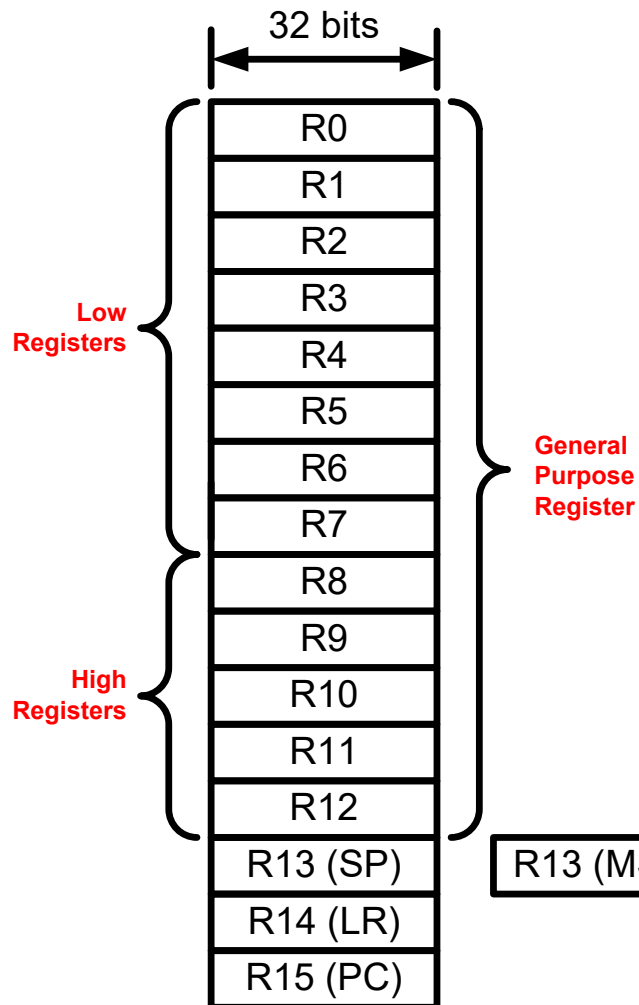
## Harvard

**Data and instructions are stored into separate memories.**

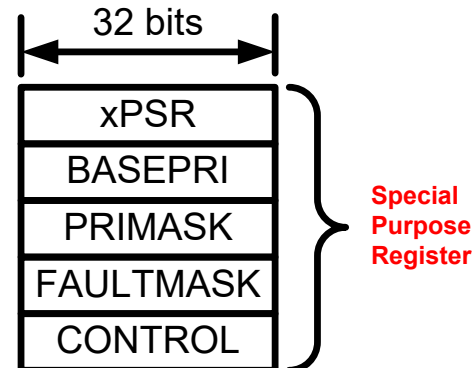




# Processor Registers

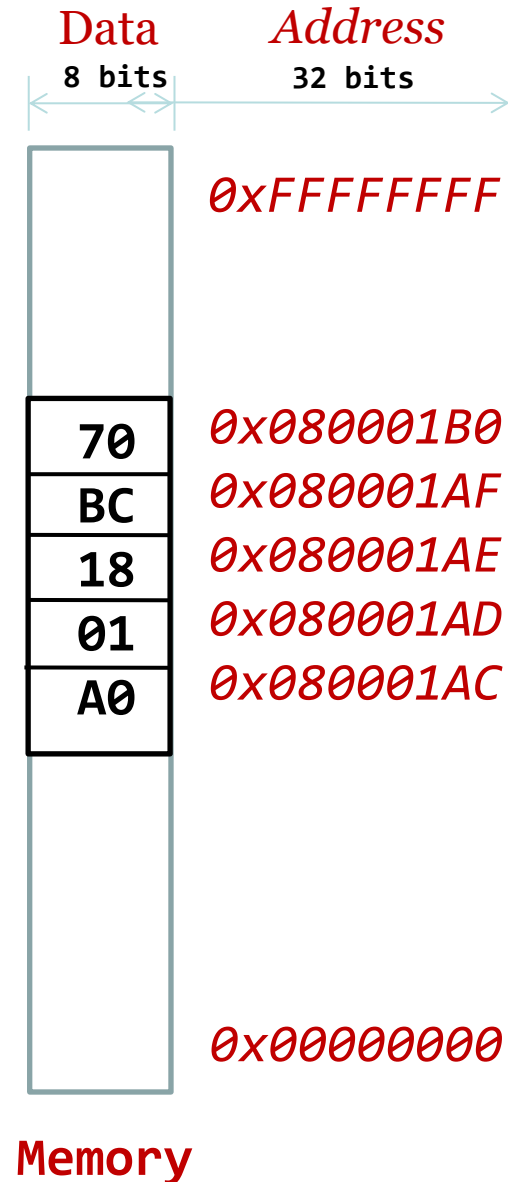


- ▶ Fastest way to read and write
- ▶ Registers are within the processor chip
- ▶ A register stores 32-bit value
- ▶ STM32L has
  - ▶ **R0-R12**: 13 general-purpose registers
  - ▶ **R13**: Stack pointer (Shadow of MSP or PSP)
  - ▶ **R14**: Link register (LR)
  - ▶ **R15**: Program counter (PC)
  - ▶ Special registers (xPSR, BASEPRI, PRIMASK, etc)



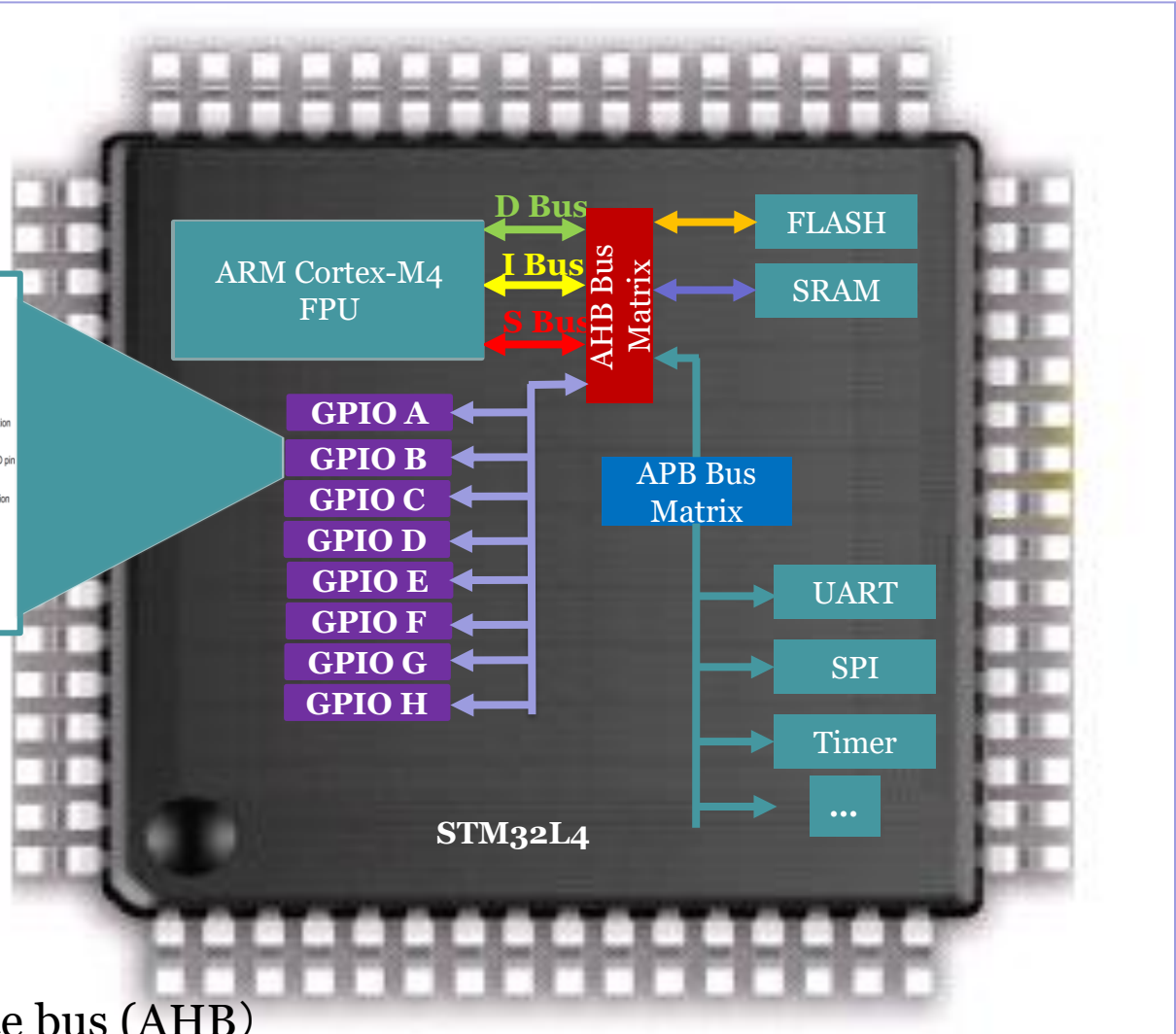
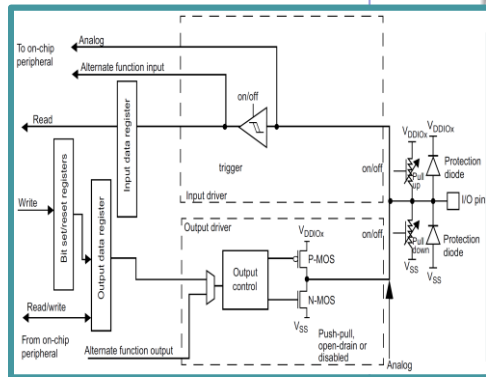
# Memory

- Memory is arranged as a series of “locations”
  - Each location has a unique “address”
  - Each location holds a byte (**byte-addressable**)
  - e.g. the memory location at address `0x080001B0` contains the byte value `0x70`, i.e., 112
- The number of locations in memory is limited
  - What is the largest memory capacity can STM32L4 support?
  - 1 Gigabyte (GB) =  $2^{30}$  bytes
  - $2^{32}$  locations → 4,294,967,296 locations!
- Values stored at each location can represent either **program data** or **program instructions**
  - e.g. the value `0x70` might be the code used to tell the processor to add two values together



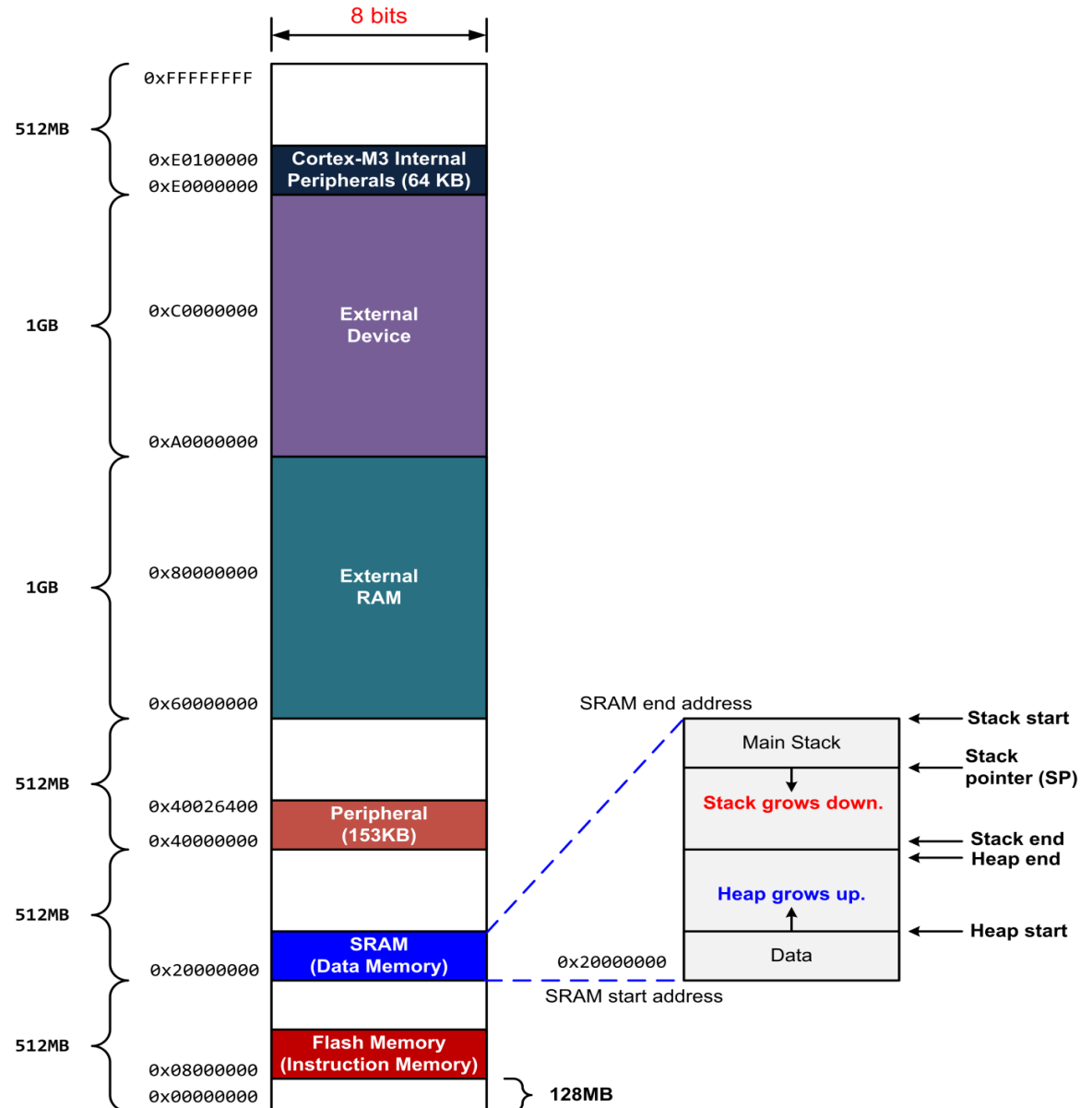


# I/O Peripherals

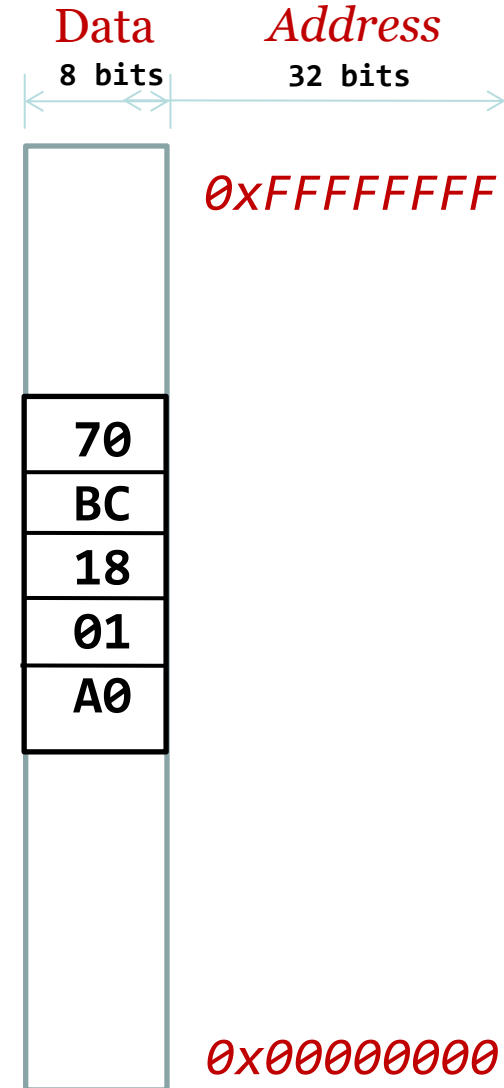
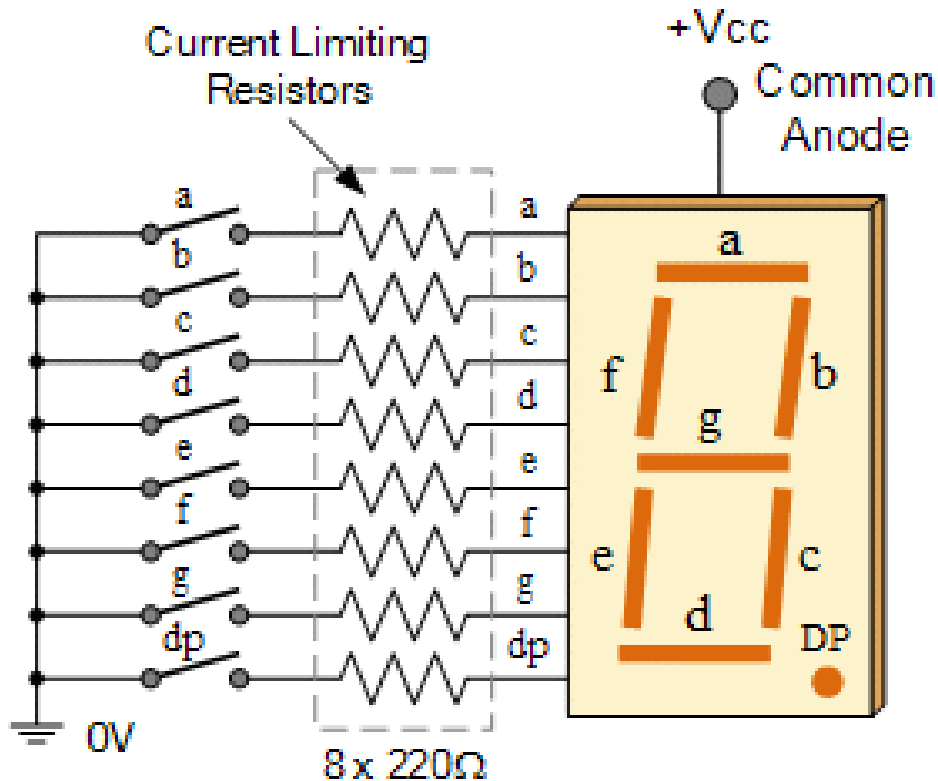


Advanced High performance bus (AHB)  
Advanced Peripheral Bus (APB).

# Memory Map



# 7-Segment Display



Memory

# Levels of Program Code

## C Program

```
int main(void){  
    int i;  
    int total = 0;  
    for (i = 0; i < 10; i++) {  
        total += i;  
    }  
    while(1); // Dead loop  
}
```

Compile

## Assembly Program

```
        MOVS r1, #0  
        MOVS r0, #0  
loop    B    check  
        ADD  r1, r1, r0  
        ADDS r0, r0, #1  
check   CMP  r0, #10  
        BLT  loop  
self    B    self
```

Assemble

## Machine Program

```
0010000100000000  
0010000000000000  
1110000000000001  
0100010000000001  
0001110001000000  
0010100000001010  
1101110011111011  
1011111100000000  
1110011111111110
```

### ▶ High-level language

- ▶ Level of abstraction closer to problem domain
- ▶ Provides for productivity and portability

### ▶ Assembly language

- ▶ Textual representation of instructions

### ▶ Hardware representation

- ▶ Binary digits (bits)
- ▶ Encoded instructions and data



# How is the program stored?

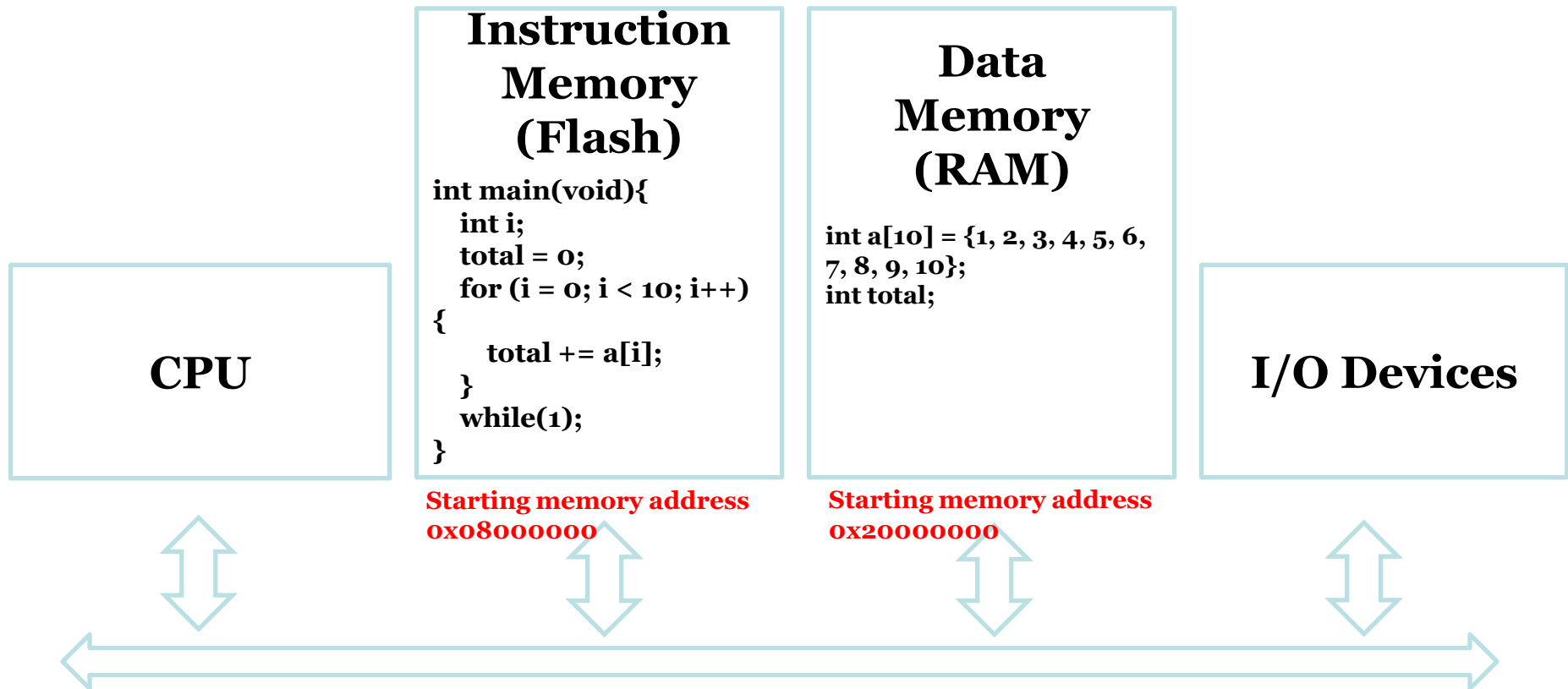
# Example:

## Calculate the Sum of an Array

```
int a[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};  
int total;  
  
int main(void){  
    int i;  
    total = 0;  
    for (i = 0; i < 10; i++) {  
        total += a[i];  
    }  
    while(1);  
}
```



# Example: Calculate the Sum of an Array



# Example: Calculate the Sum of an Array

## Instruction Memory (Flash)

```
int main(void){
    int i;
    total = 0;
    for (i = 0; i < 10; i++)
    {
        total += a[i];
    }
    while(1);
}
```

Starting memory address  
0x08000000

```
0010 0001 0000 0000
0100 1010 0000 1000
0110 0000 0001 0001
0010 0000 0000 0000
1110 0000 0000 1000
0100 1001 0000 0111
1111 1000 0101 0001
0001 0000 0010 0000

0100 1010 0000 0100
0110 1000 0001 0010
0100 0100 0001 0001
0100 1010 0000 0011
0110 0000 0001 0001
0001 1100 0100 0000
0010 1000 0000 1010
1101 1011 1111 0100
1011 1111 0000 0000
1110 0111 1111 1110
```



```
MOVS r1, #0x00
LDR r2, = total_addr
STR r1, [r2, #0x00]
MOVS ro, #0x00
B Check
Loop: LDR r1, = a_addr
LDR r1, [r1, ro, LSL #2]
LDR r2, = total_addr
LDR r2, [r2, #0x00]
ADD r1, r1, r2
LDR r2, = total_addr
STR r1, [r2, #0x00]
ADDS ro, ro, #1
Check: CMP ro, #0x0A
BLT Loop
NOP
Self: B Self
```

## Example: Calculate the Sum of an Array

### Data Memory (RAM)

```
int a[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
int total;
```

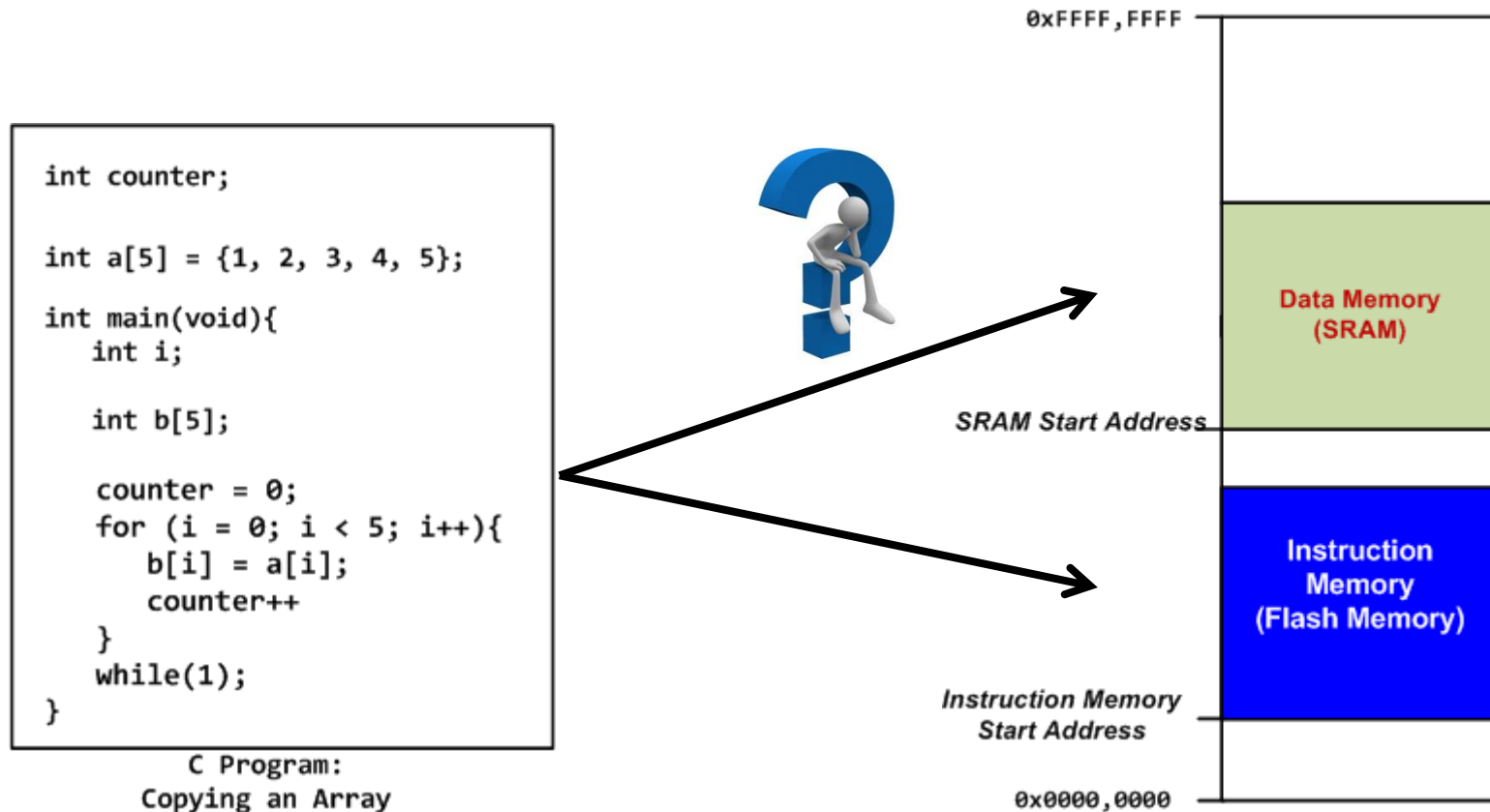
Assume the starting memory address of  
the data memory is 0x20000000

|            |        |                   |
|------------|--------|-------------------|
| 0x20000000 | 0x0001 | a[0] = 0x00000001 |
| 0x20000002 | 0x0000 |                   |
| 0x20000004 | 0x0002 | a[1] = 0x00000002 |
| 0x20000006 | 0x0000 |                   |
| 0x20000008 | 0x0003 | a[2] = 0x00000003 |
| 0x2000000A | 0x0000 |                   |
| 0x2000000C | 0x0004 | a[3] = 0x00000004 |
| 0x2000000E | 0x0000 |                   |
| 0x20000010 | 0x0005 | a[4] = 0x00000005 |
| 0x20000012 | 0x0000 |                   |
| 0x20000014 | 0x0006 | a[5] = 0x00000006 |
| 0x20000016 | 0x0000 |                   |
| 0x20000018 | 0x0007 | a[6] = 0x00000007 |
| 0x2000001A | 0x0000 |                   |
| 0x2000001C | 0x0008 | a[7] = 0x00000008 |
| 0x2000001E | 0x0000 |                   |
| 0x20000020 | 0x0009 | a[8] = 0x00000009 |
| 0x20000022 | 0x0000 |                   |
| 0x20000024 | 0x000A | a[9] = 0x0000000A |
| 0x20000026 | 0x0000 |                   |
| 0x20000028 | 0x0000 | total= 0x00000000 |
| 0x2000002A | 0x0000 |                   |

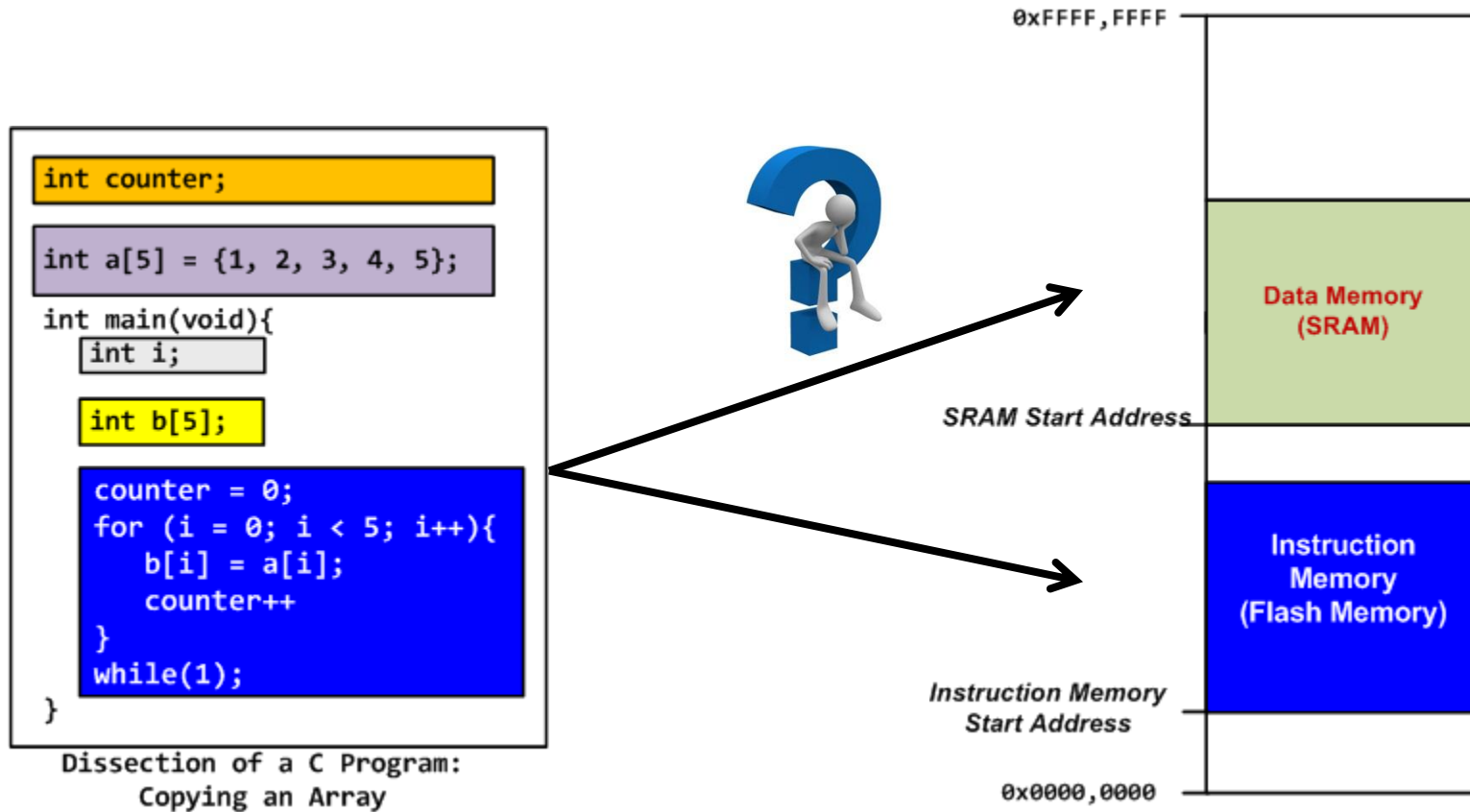
Memory  
address  
in bytes

Memory  
content

# Loading Code and Data into Memory

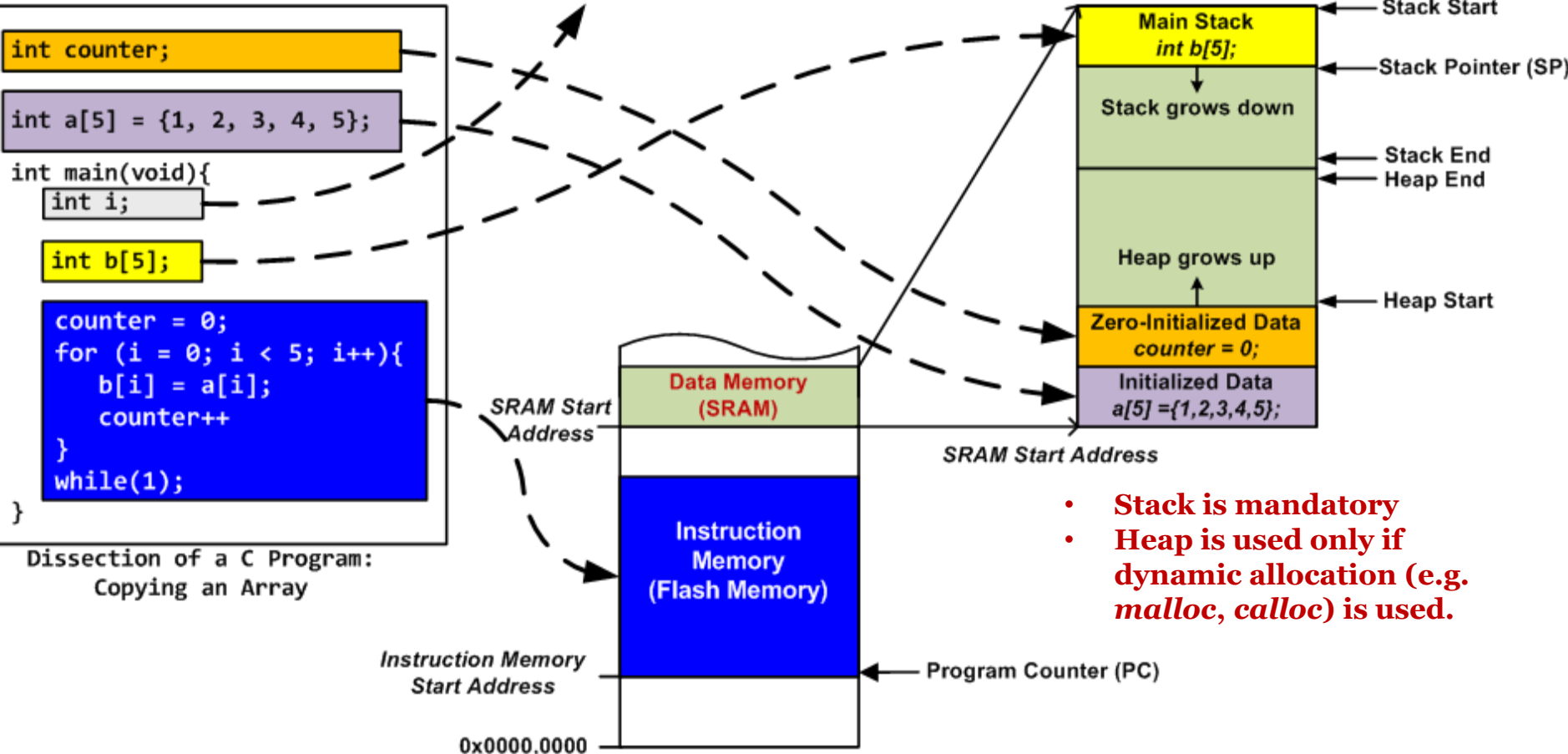


# Loading Code and Data into Memory



# Loading Code and Data into Memory

To improve performance, some variables are not stored in memory.  
Variable *i* will be stored in a register.



- **Stack is mandatory**
- **Heap is used only if dynamic allocation (e.g. *malloc*, *calloc*) is used.**



# How does the program run?

# See a Program Runs

## C Code

```
int main(void){  
    int a = 0;  
    int b = 1;  
    int c;  
    c = a + b;  
    return 0;  
}
```

compiler



## Assembly Code

```
MOVS r1, #0x00 ; int a = 0  
MOVS r2, #0x01 ; int b = 1  
ADDS r3, r1, r2 ; c = a + b  
MOVS r0, 0x00 ; set return value  
BX lr ; return
```

assembler



## Machine Code

```
0010000100000000  
00100010000000001  
0001100010001011  
0010000000000000  
0100011101110000
```

In Binary

```
2100  
2201  
188B  
2000  
4770
```

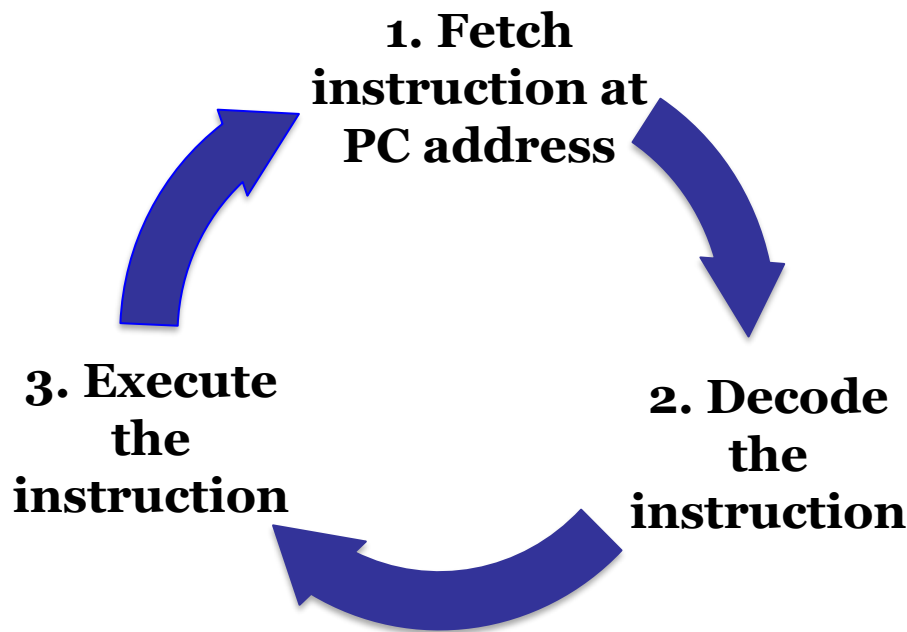
In Hex

```
; MOVS    r1, #0x00  
; MOVS    r2, #0x01  
; ADDS    r3, r1, r2  
; MOVS    r0, #0x00  
; BX      lr
```



# Program Execution

- Program Counter (PC)** is a register that holds the memory address of the next instruction to be fetched from the memory.



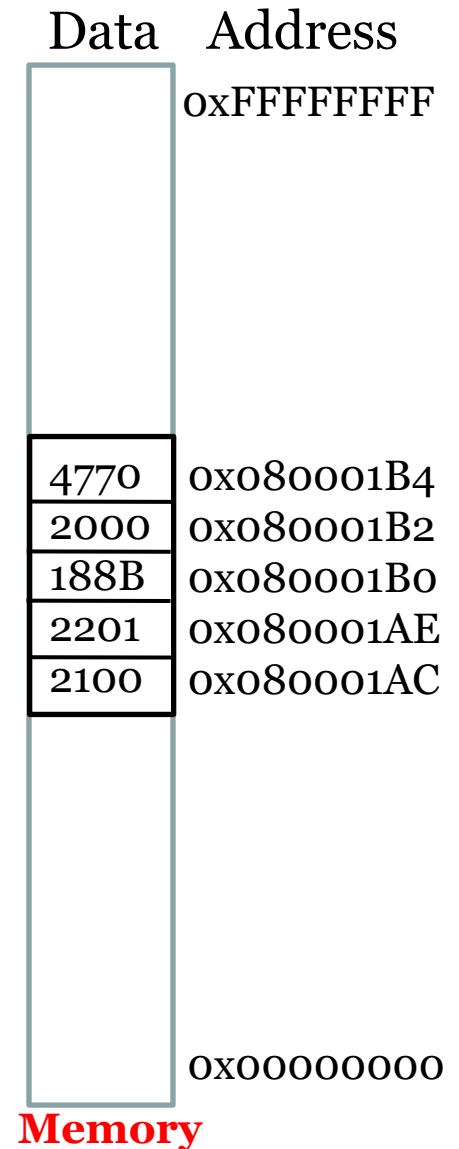
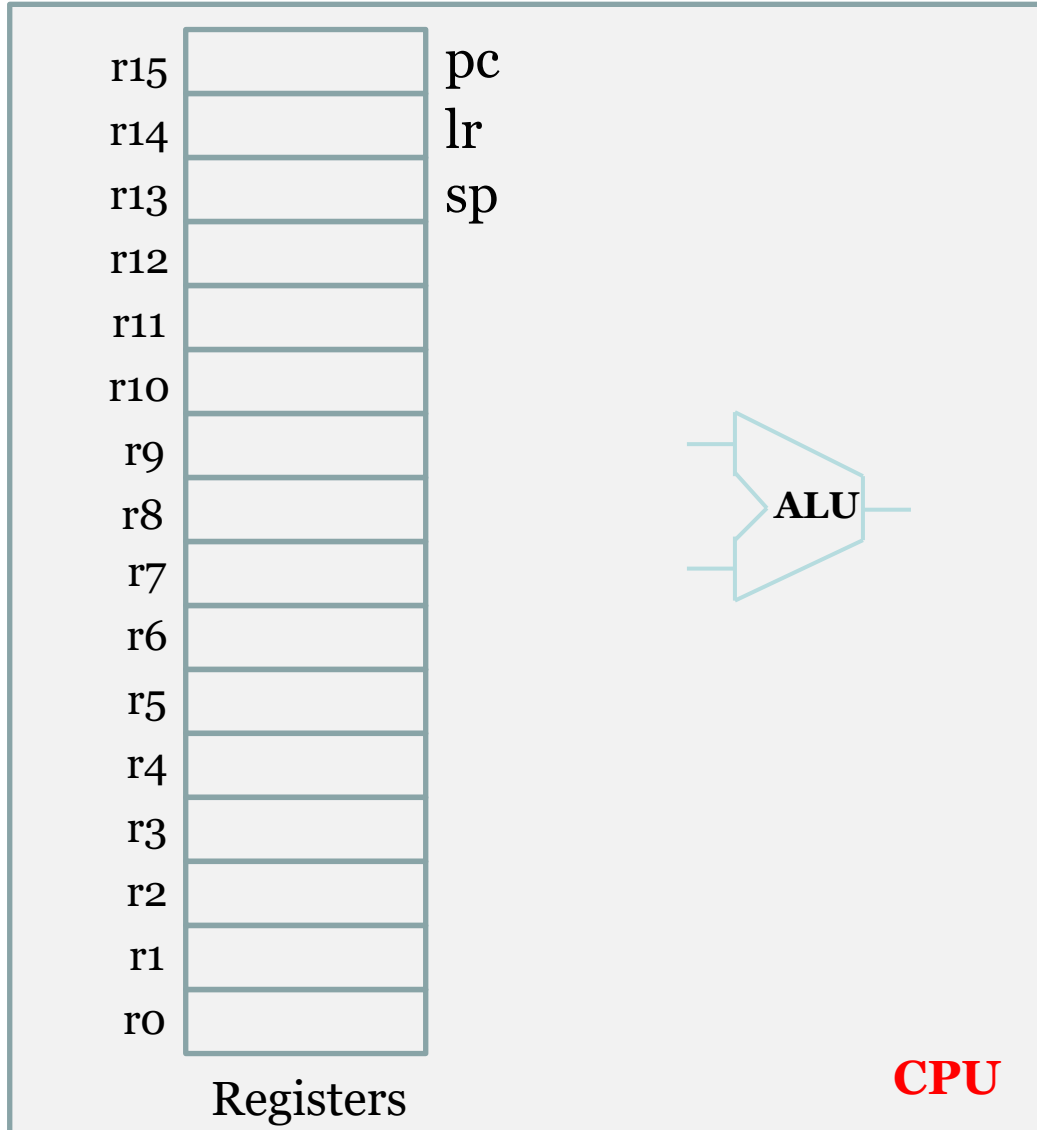
Memory Address

|      |            |
|------|------------|
| 4770 | 0x080001B4 |
| 2000 | 0x080001B2 |
| 188B | 0x080001B0 |
| 2201 | 0x080001AE |
| 2100 | 0x080001AC |

PC →

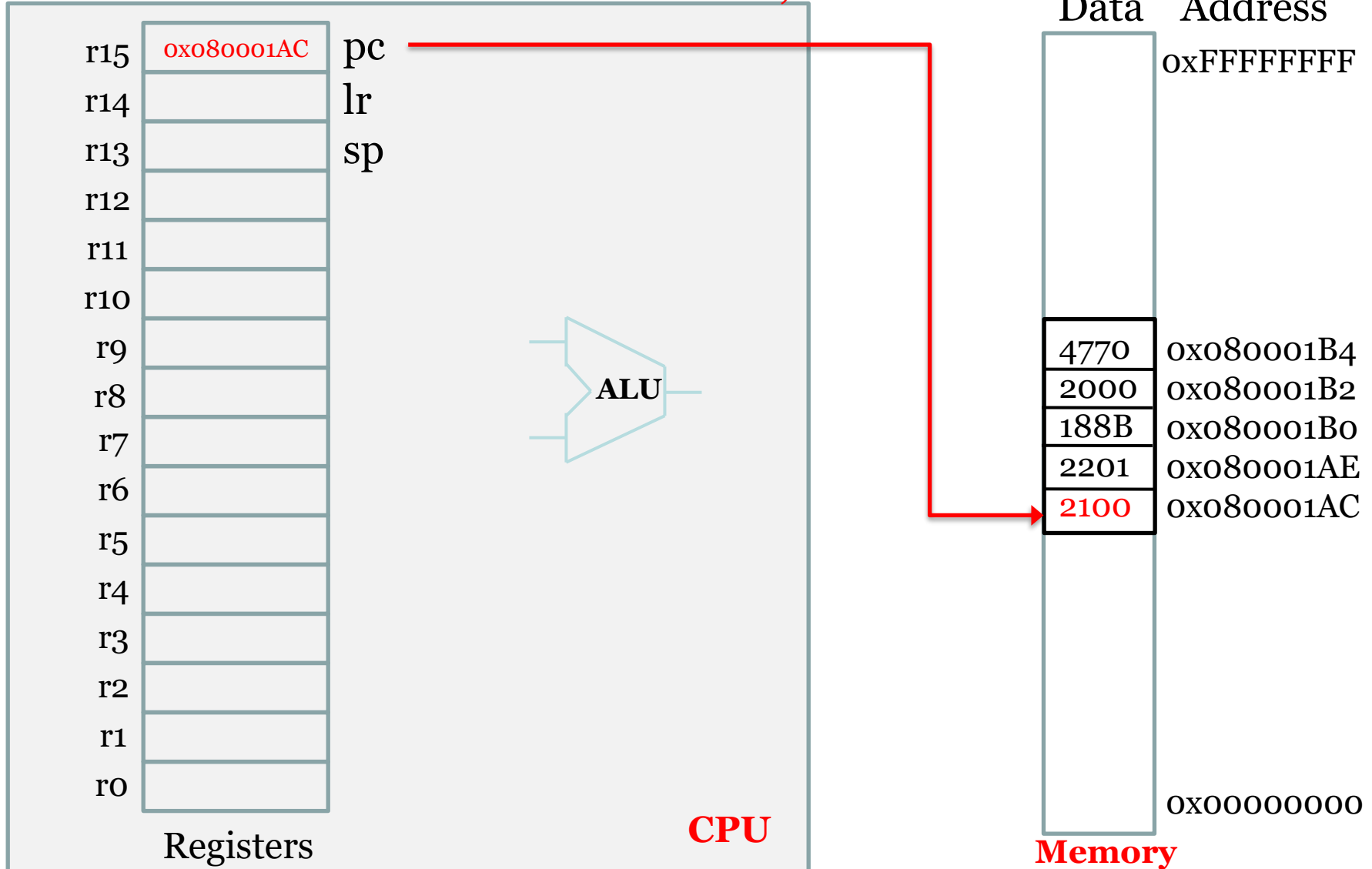
PC = 0x080001B0  
Instruction = 188B or  
2000188B or 8B180020

## Machine codes are stored in memory



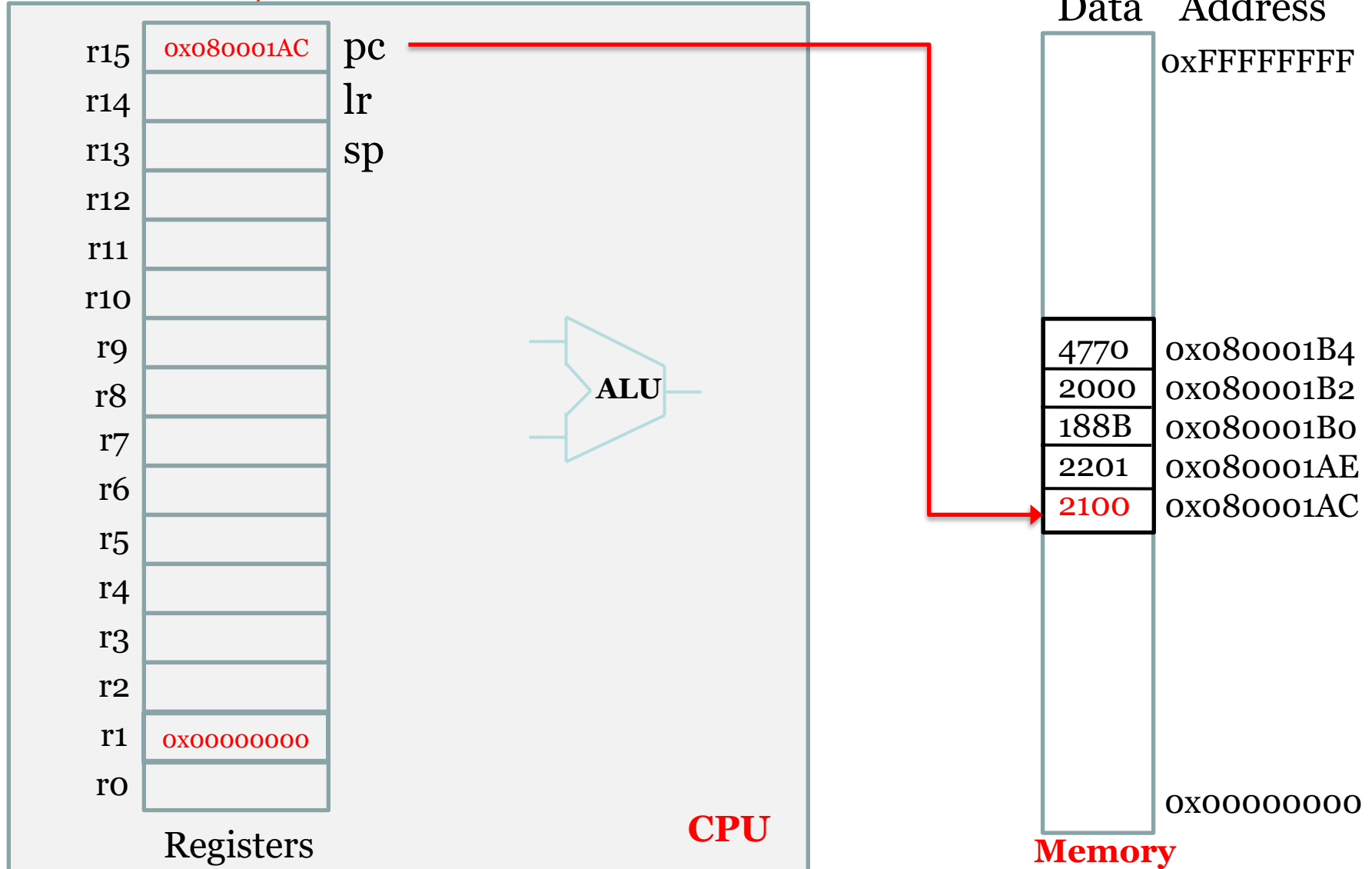
**Fetch Instruction: pc = 0x08001AC**

**Decode Instruction: 2100 = MOVs r1, #0x00**



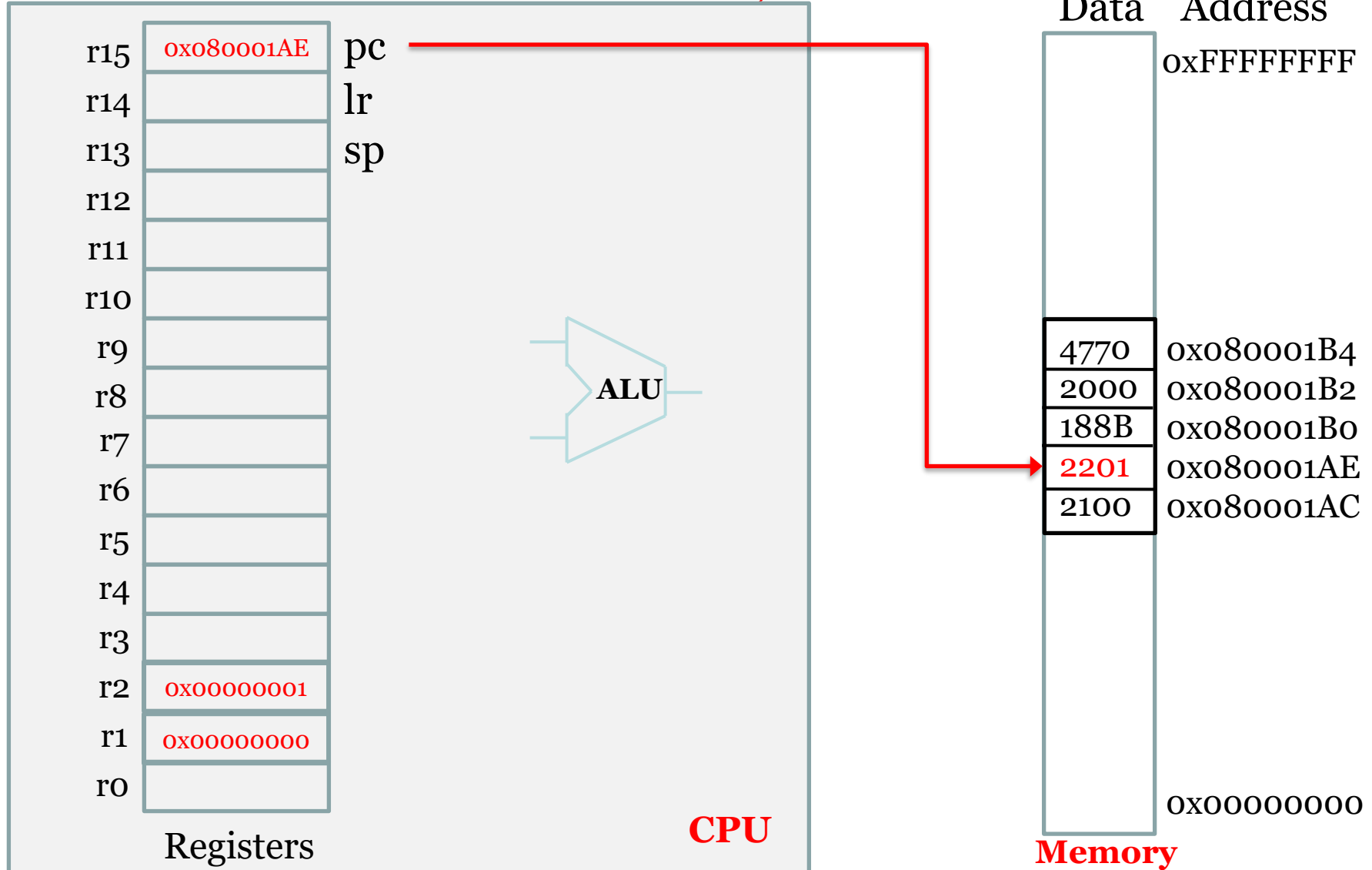
## Execute Instruction:

**MOVS r1, #0x00**



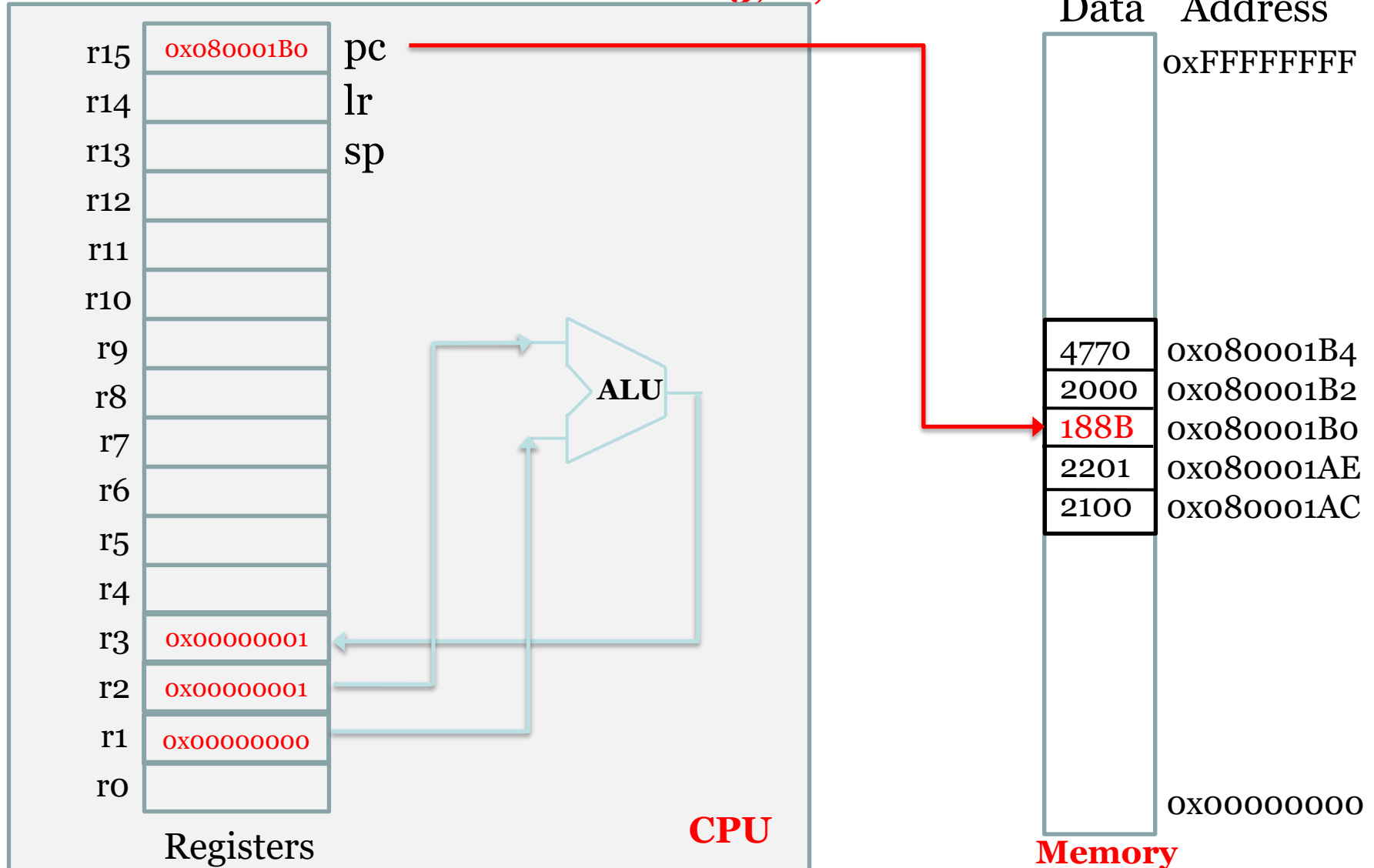
**Fetch Next Instruction:  $pc = pc + 2$**

**Decode & Execute: 2201 = **MOVS** r2, #0x01**



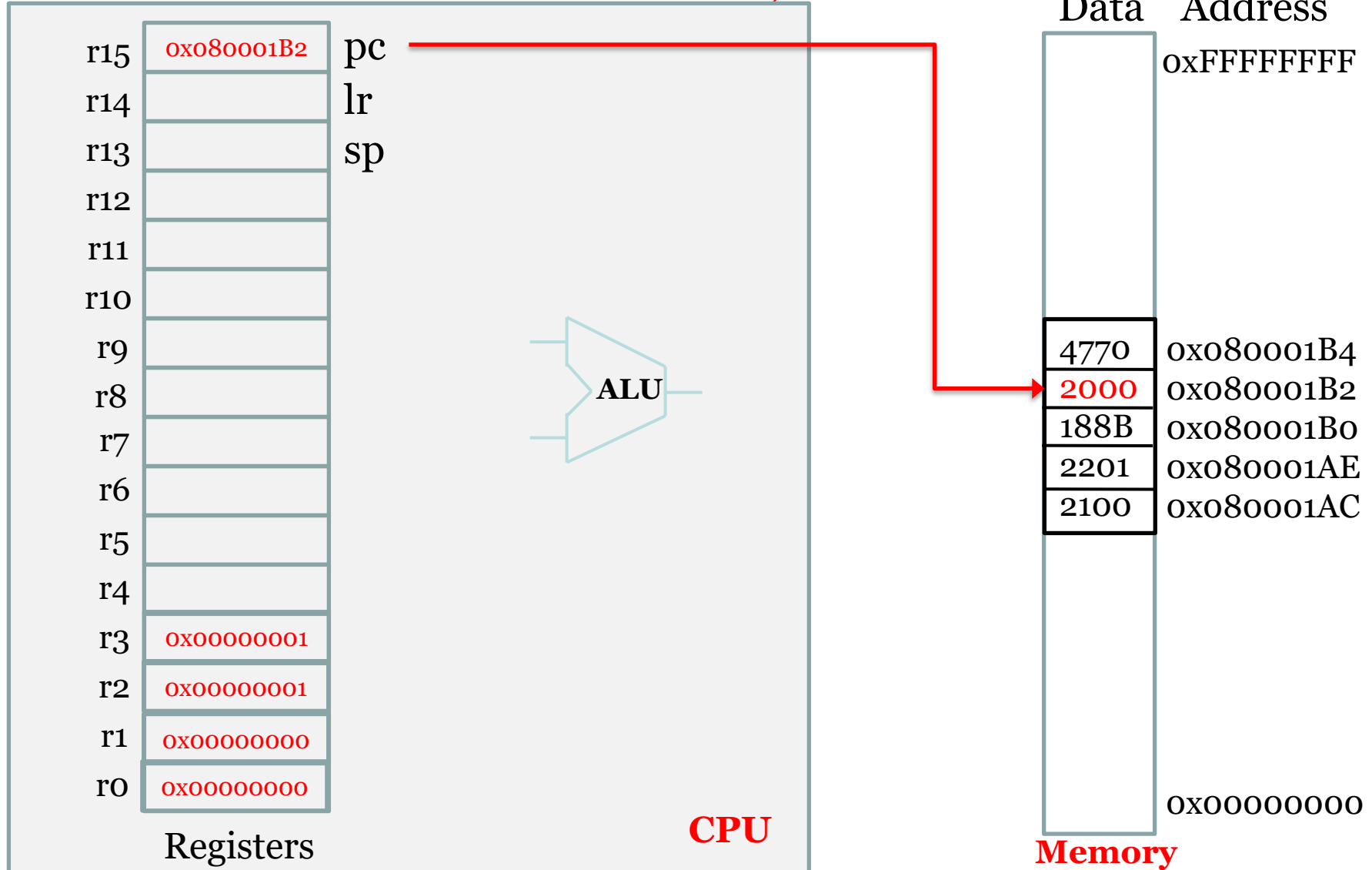
**Fetch Next Instruction:  $pc = pc + 2$**

**Decode & Execute: 188B = **ADDS** r3, r1, r2**



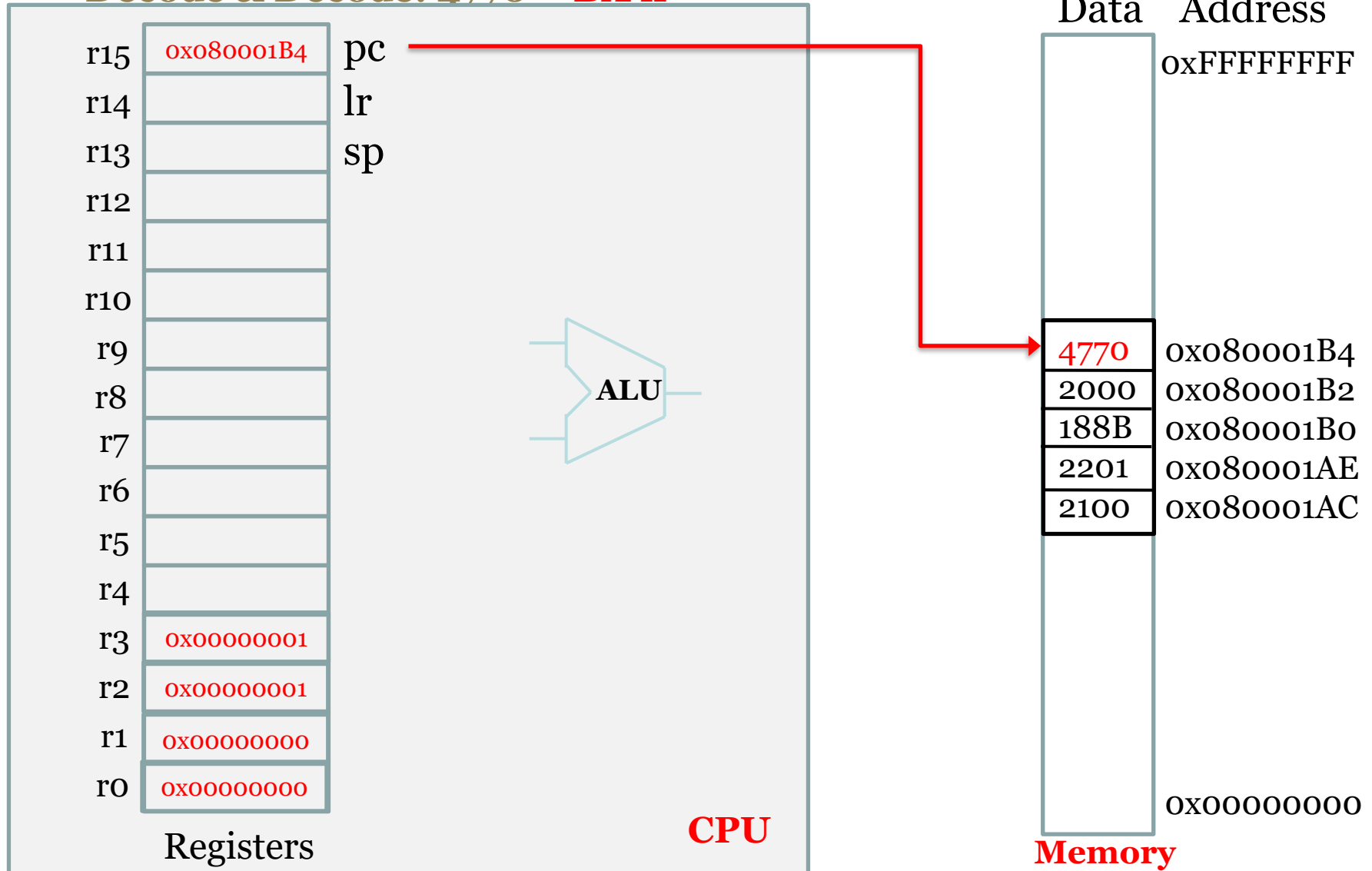
Fetch Next Instruction:  $pc = pc + 2$

Decode & Execute:  $2000 = \text{MOVS } r0, \#0x00$



Fetch Next Instruction:  $pc = pc + 2$

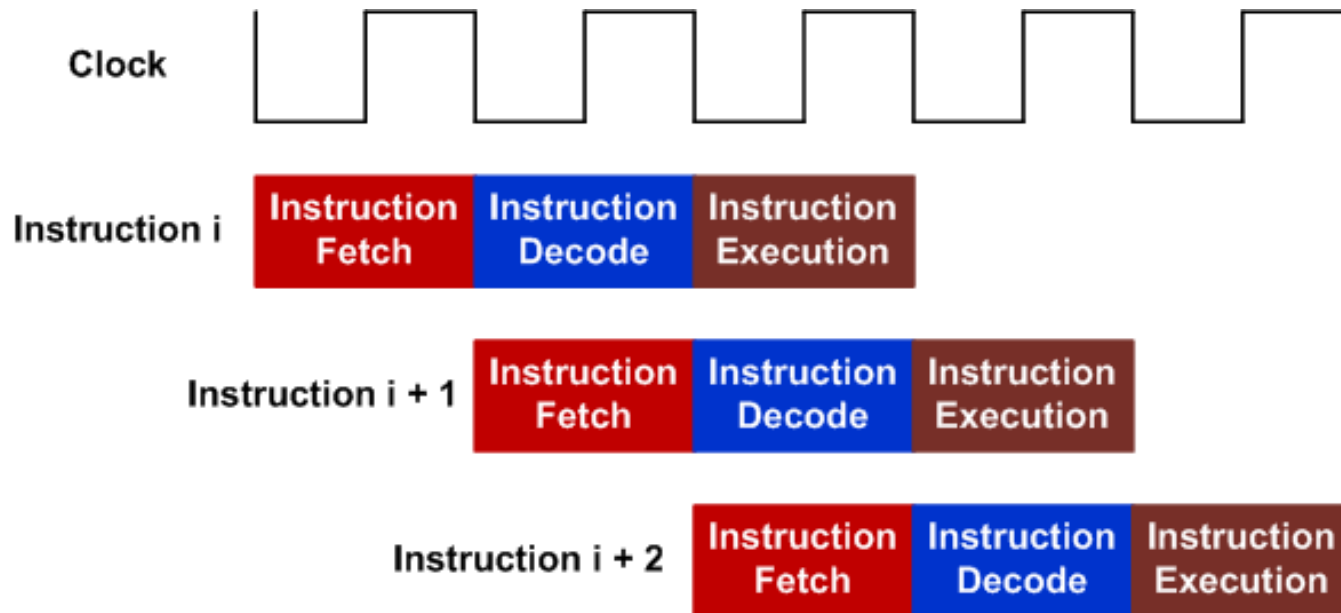
Decode & Decode: 4770 = **BX** lr





# Three-state pipeline: Fetch, Decode, Execution

- **Pipelining** allows hardware resources to be fully utilized
- One 32-bit instruction or two 16-bit instructions can be fetched.



## Pipeline of 32-bit instructions

# Three-state pipeline: Fetch, Decode, Execution

- **Pipelining** allows hardware resources to be fully utilized
- One 32-bit instruction or two 16-bit instructions can be fetched.

